

SELENIUM WEEK 3 -DEMO REPORT

Chandana B.

User Story

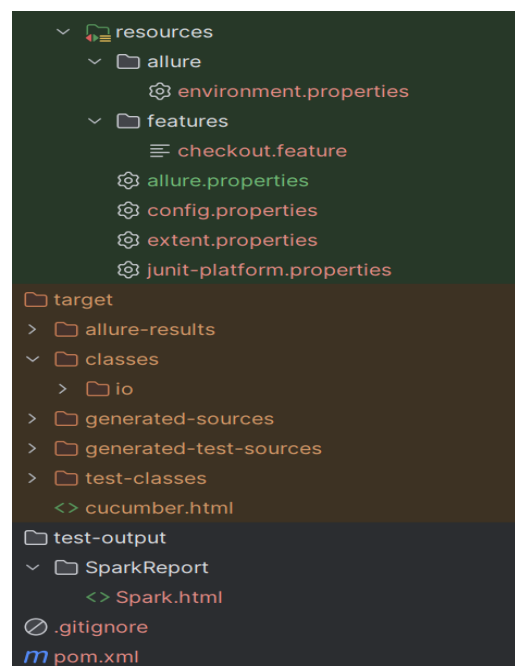
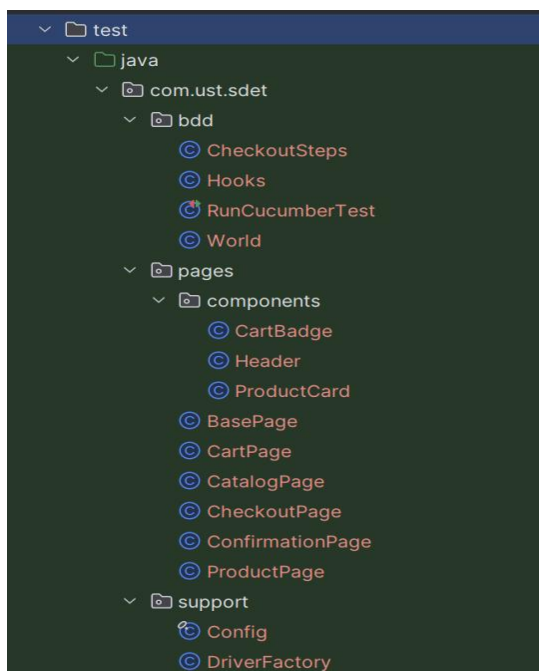
Title: Purchase a Product from the Retail Store

As a shopper, I want to search for a product, view its details, add it to my cart, and complete the checkout process So that I can successfully purchase products through the online retail application.

Automated Application flow

Open Application → Navigate to Products → Search Product → Open Product Details → Add Product to Cart → Validate Cart Badge → Open Cart → Validate Cart Items → Proceed to Checkout → Place Order → Order Confirmation

Folder Structure



Framework Design approach

I followed Page Object Model . The framework is organized into:

1.Pages: Each page of the application is represented using a class.The pages we used are CatalogPage,ProductPage,CartPage,CheckoutPage,ConfirmationPage.Each page contains locators and actions that are related to that particular page.

2.Components: The elements of UI which are used in multiple pages. Components we used for the project are mainly Header(used to navigate to pages),CartBadge(To see the count of product added to cart),ProductCard(Opening the product details).

3.Base Page:This page mainly consists of actions that are performed by selenium to interact with elements in the application.Methods like click(),type(),visbleElements(),text() are used for the project .

4.PicoContainer:It is used to share the objects used across the teststeps.We have used world file here.

5.DriverFactory: It is created for the use of Browser creation, Browser configuration and headless support.

6.Feature files: The entire user journey and scenarios to test the application are written in plain and readable English.

7.Hooks: The steps which needs to be performed at every stage before or after (@Before and @After).

Property files

The files used are :

1.config.properties: Created and used to store all configurations and executions.

Contents of the file:

```
browser=chrome  
headless=false  
baseUrl=http://localhost:5173  
timeout=10
```

2.environment.properties: Created and used to include all execution related environments

Contents of the file:

```
Browser=Chrome  
Headless=true  
BaseURL=http://localhost:5173  
Build=local  
Framework=SeleniumCucumberPOM  
Java=21
```

3.allure.properties: Created and used to configure allure reporting.

Contents of the file:

`allure.results.directory=target/allure-results`

4.extent.properties: Created and used to configure Extent reports.

Contents of the file:

`extent.reporter.spark.start=true`

`extent.reporter.spark.out=test-output/SparkReport/Spark.html`

`extent.reporter.spark.base64imagesrc=true`

`screenshot.dir=test-output/screenshots/`

`screenshot.rel.path=../screenshots/`

`systeminfo.Browser=Chrome(headless)`

`systeminfo.BaseURL=http://localhost:5173`

`systeminfo.Build=local`

`systeminfo.OS=local workstation`

5.junit-platform.properties:Created and used to centralise all the reporting and cucumber related executions.

Contents of the file:

`cucumber.glue=com.ust.sdet.bdd`

`cucumber.plugin=pretty,html:target/cucumber.html,io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm,com.aventstack.extentreports.cucumber.adapter.ExtentCucumberAdapter:`

`#cucumber.filter.tags=@smoke,@regression,@negative`

`cucumber.publish.quiet=true`

The feature file design

Contents of the file:

Feature: Catalog checkout

As a shopper

I want to find products, add them to my cart, and complete an order

So that the retail purchase journey is covered in shared language

Background:

Given the catalog is open

@smoke

Scenario: Buy a single product end to end

When I search for "travel backpack"

And I open the first matching product

And I add the product to the cart

Then the cart badge shows 1

When I open the cart

Then the cart has 1 line items

When I proceed to checkout

And I place the order

Then the order is confirmed

@smoke

Scenario: Cart badge reflects the number of items added

When I search for "travel backpack"

And I open the first matching product

And I add the product to the cart

Then the cart badge shows 1

@regression

Scenario Outline: Buy a "<product>" end to end

When I search for "<product>"

And I open the first matching product

And I add the product to the cart

Then the cart badge shows <count>

When I open the cart

Then the cart has <count> line items

When I proceed to checkout

And I place the order

Then the order is confirmed

Examples:

product	count
running shoes	1
travel backpack	1
headphones	1

@regression

Scenario: A fresh cart is empty

When I open the cart

Then the cart has 0 line items

@negative

Scenario: Deliberate failure to demonstrate screenshot capture

When I search for "travel backpack"

And I open the first matching product

And I add the product to the cart

Then the cart badge shows 999

Explanation:

Feature section is a the functionality of what we are going to test.

Background section consists of the pre conditions required to perform the test.

Tags:

@smoke: The tag is used to perform the tests which have the scenario of the critical functionality.

@regression: The tag is used to perform the tasks which should be performed continuously

@negative: The tag is used to perform the negative testing where the test contains invalid details.

Scenario-outline: It is used when steps should be performed on multiple inputs , so we use placeholders instead of directly defined.

Scenario	Type	Description
Buy a Single Product End-to-End	@smoke	Validates the complete purchase workflow from product search to order confirmation.
Cart Badge Validation	@smoke	Validates the complete purchase workflow from product search to order confirmation.
Buy Product End-to-End	@regression	Validates the purchase workflow using multiple products through a Scenario Outline.
Empty Cart Validation	@regression	Verifies that a newly opened cart contains no items.
ScreenshotCapture Validation	@negative	Validates screenshot capture and reporting when a test fails.

Commands To Run

1.mvn test -Dheadless=true

What we expect:

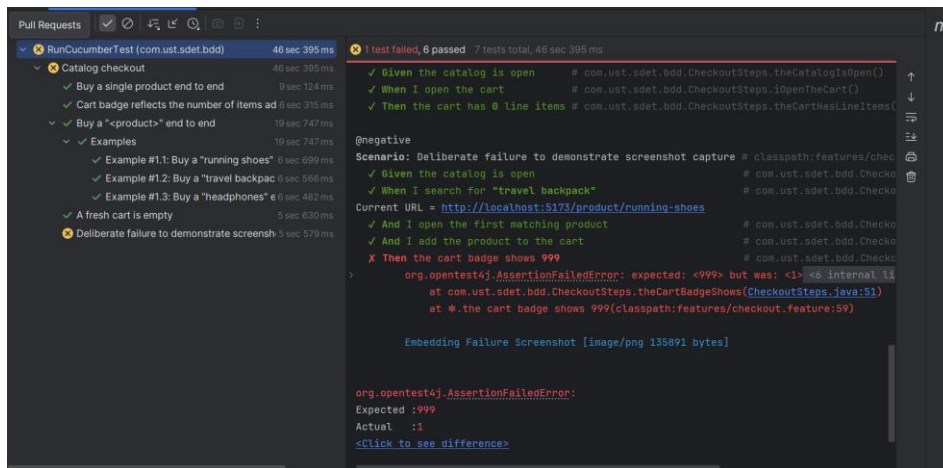
```

RunCucumberTest (com.ust.sdet.bdd) 46 sec 395 ms
  Catalog checkout 46 sec 395 ms
    Buy a single product end to end 9 sec 124 ms
    Cart badge reflects the number of items added 6 sec 315 ms
  Buy a "<product>" end to end 19 sec 747 ms
    Examples 19 sec 747 ms
      Example #1.1: Buy a "running shoes" 6 sec 699 ms
      Example #1.2: Buy a "travel backpack" 6 sec 566 ms
      Example #1.3: Buy a "headphones" 6 sec 482 ms
      A fresh cart is empty 5 sec 630 ms
      Deliberate failure to demonstrate screenshot capture 5 sec 579 ms

1 test failed, 6 passed 7 tests total, 46 sec 395 ms
C:\Users\304525\jdk\ms-21.0.11\bin\java.exe ...

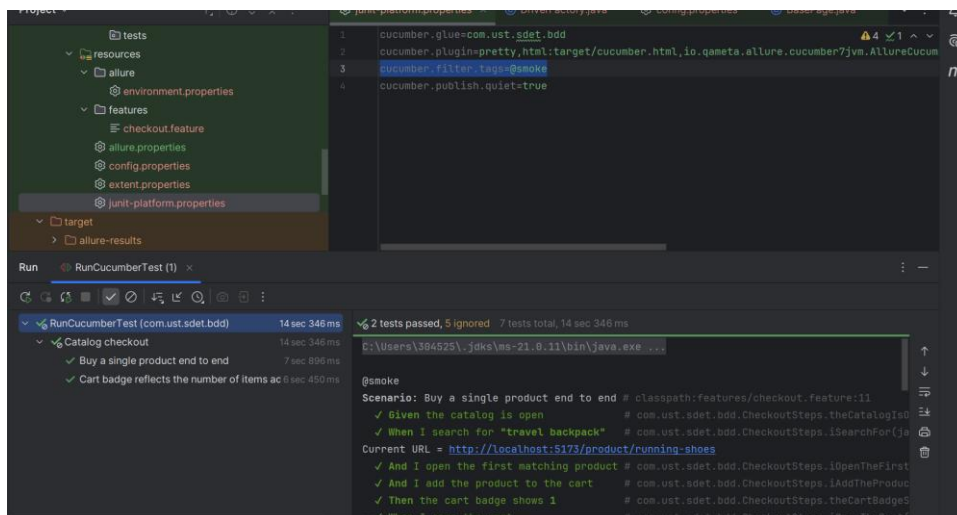
@smoke
Scenario: Buy a single product end to end # classpath:features/checkout.feature:11
  Given the catalog is open # com.ust.sdet.bdd.CheckoutSteps.theCatalogIsOpen
  When I search for "travel backpack" # com.ust.sdet.bdd.CheckoutSteps.iSearchForProduct
Current URL = http://localhost:5173/product/running-shoes
  And I open the first matching product # com.ust.sdet.bdd.CheckoutSteps.iOpenTheFirstProduct
  And I add the product to the cart # com.ust.sdet.bdd.CheckoutSteps.iAddTheProduct
  Then the cart badge shows 1 # com.ust.sdet.bdd.CheckoutSteps.theCartBadgeShows
  When I open the cart # com.ust.sdet.bdd.CheckoutSteps.iOpenTheCart
  Then the cart has 1 line items # com.ust.sdet.bdd.CheckoutSteps.theCartHasLineItems
  When I proceed to checkout # com.ust.sdet.bdd.CheckoutSteps.iProceedToCheckout
  And I place the order # com.ust.sdet.bdd.CheckoutSteps.iPlaceTheOrder
  Then the order is confirmed # com.ust.sdet.bdd.CheckoutSteps.theOrderIsConfirmed

@smoke
Scenario: Cart badge reflects the number of items added # classpath:features/checkout.feature:12
  Given the catalog is open # com.ust.sdet.bdd.CheckoutSteps.theCatalogIsOpen
  When I search for "travel backpack" # com.ust.sdet.bdd.CheckoutSteps.iSearchForProduct
Current URL = http://localhost:5173/product/running-shoes
  And I open the first matching product # com.ust.sdet.bdd.CheckoutSteps.iOpenTheFirstProduct
  And I add the product to the cart # com.ust.sdet.bdd.CheckoutSteps.iAddTheProduct
  Then the cart badge shows 1 # com.ust.sdet.bdd.CheckoutSteps.theCartBadgeShows
  
```



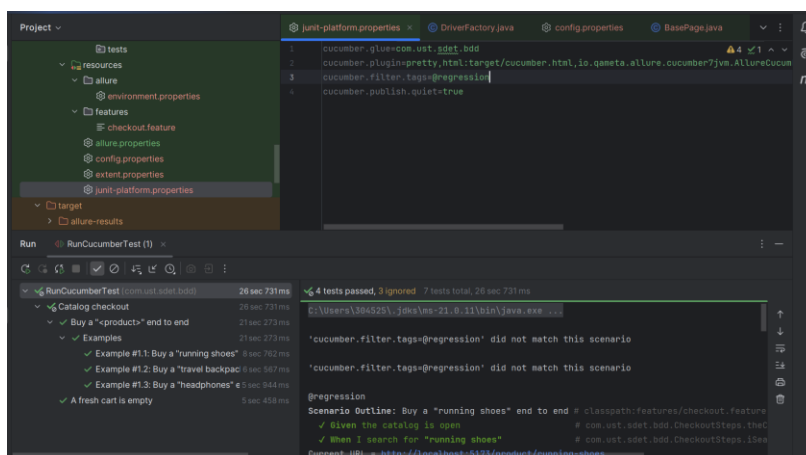
2. mvn test -Dcucumber.filter.tags="@smoke"

What we expect:



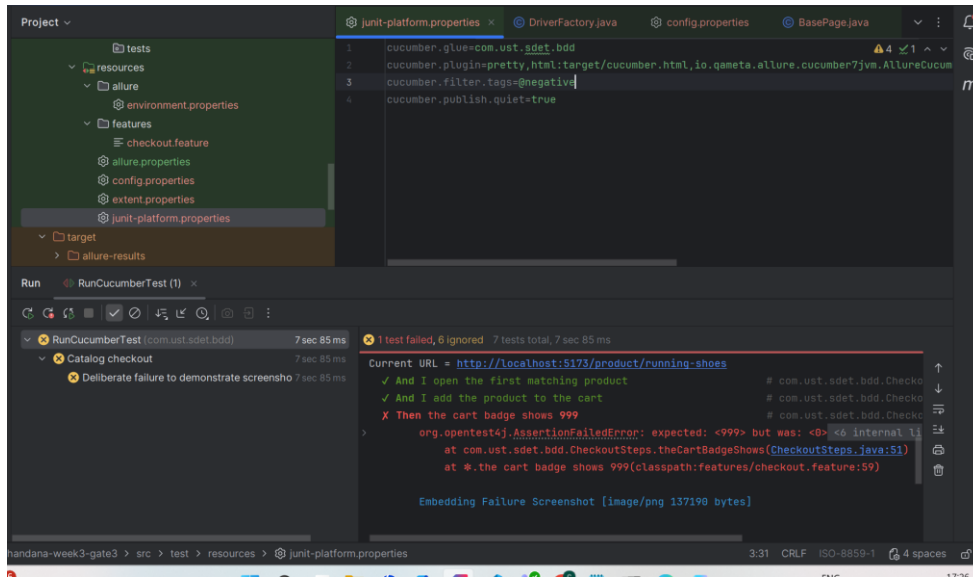
3. mvn test -Dcucumber.filter.tags="@regression"

What we expect:



4. mvn test -Dcucumber.filter.tags="@negative"

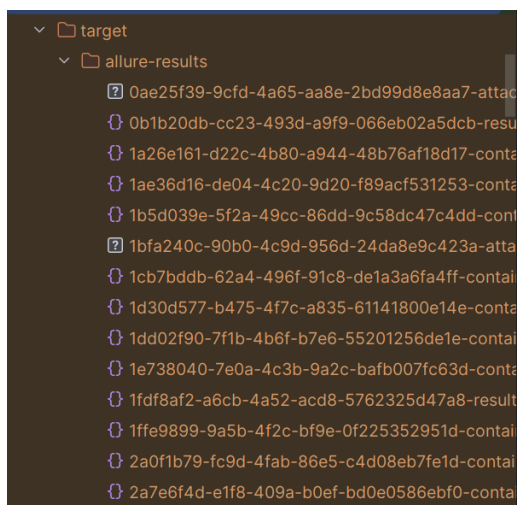
What we expect:



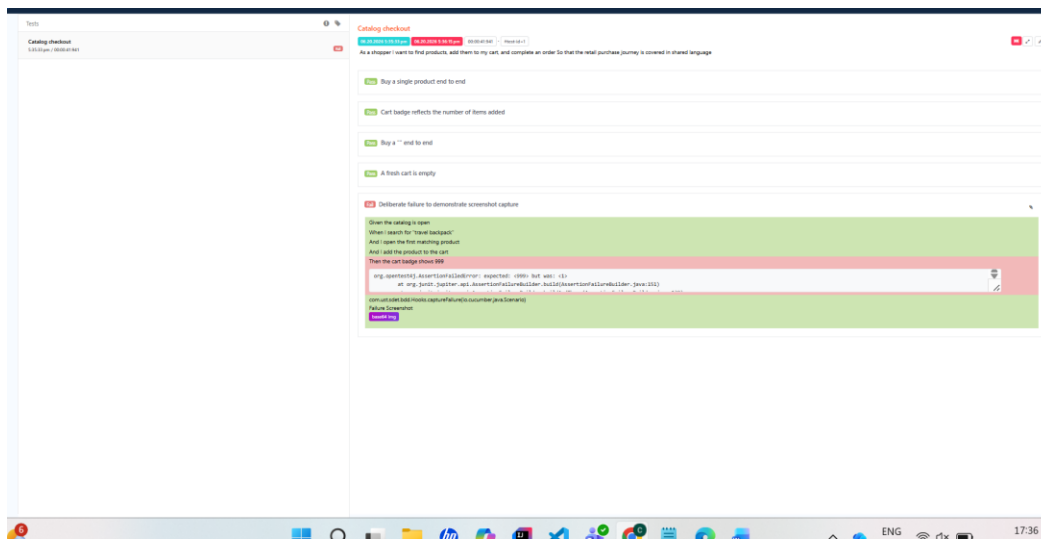
Note: In terminal my commands are not running because of SSL certificate issue



5. allure serve target/allure-results



6. Spark report



Reason for the failure of negative test case:

We have the scenario like:

@negative

Scenario: Deliberate failure to demonstrate screenshot capture

When I search for "travel backpack"

And I open the first matching product

And I add the product to the cart

Then the cart badge shows 999

So when I add the product to cart the cart badge shows 1 not 999. So the test will be failed and captures the screenshot.

